INT161 · Basic Backend Development

Database Connection

เชื่อม Node.js กับ MySQL ด้วย pool ป้องกัน SQL injection ด้วย placeholder และเขียน repository CRUD

Database Setup | mysql2 and the Pool | Prepared Statements | The Repository | Reading the Body and Writing the Response | CRUD over HTTP | อภิธานศัพท์

01

Database Setup

คลาสที่แล้วข้อมูลอยู่ใน array ในหน่วยความจำ วันนี้ย้ายไปอยู่ใน database จริง เป้าหมาย: ทำ **CRUD** กับ MySQL จาก **Node**

- **back end** ทำหน้าที่คนกลาง **front end** ส่ง method มา back end แปลงเป็น SQL ส่งให้ **DBMS** แล้วส่งผลกลับเป็น **JSON**
- ขั้นที่สำคัญคือ **connection** ถ้าไม่มี connection ก็ส่ง SQL ไม่ได้ เชื่อมต่อได้แล้ว CRUD ก็ง่าย
- ไม่บังคับ: เครื่องมือ database ใน WebStorm ให้เชื่อมต่อและรัน SQL ในตัว editor ได้ ไม่ต้องสลับไป Workbench host **localhost**, **port** 3306, user และ password กด Test Connection ถ้า success ก็พร้อม
- เชื่อมต่อโดยยังไม่ระบุชื่อ database ก่อน เพราะกำลังจะสร้างใหม่

Create the database and table

```sql
create database sampledb;
use sampledb;
create table subjects (
  id int primary key auto_increment,
  subject_code varchar(8) not null unique,
  subject_title varchar(60),
  credit int not null
);
insert into subjects (subject_code, subject_title, credit)
values ('INT 100', 'IT Fundamentals', 3),
       ('INT 101', 'Programming Fundamentals', 3),
       ('INT 102', 'Web Technology', 1);
```

- id เป็น **surrogate key**: จำนวนเต็ม **primary key** และ **AUTO_INCREMENT** database นับให้เอง จึงไม่ต้องส่ง id ตอน INSERT และไม่ซ้ำ
- subject_code เป็น NOT NULL และ UNIQUE ถ้าใส่ INT 200 ซ้ำจะถูกปฏิเสธ
- CHAR หรือ VARCHAR ถ้าทุกค่ามี 8 ตัวอักษรพอดี CHAR(8) เหมาะกว่า VARCHAR มีภาระเพิ่มเพราะรองรับความยาวที่ต่างกัน
- ถ้า copy SQL จาก PowerPoint อาจติดสัญลักษณ์ขึ้นบรรทัดใหม่ที่มองไม่เห็น ให้ find and replace เป็นค่าว่าง แล้วจัดรูปแบบ code ใหม่

> ⚠ **ระวัง WHERE**
>
> UPDATE และ DELETE ที่ไม่มี WHERE จะเปลี่ยนทุกแถวในตาราง อาจารย์เคยเห็นข้อมูลหายแบบนี้ เช็ค WHERE ก่อนรันเสมอ

02

mysql2 and the Pool

Node ต้องมี **driver** ของตัวเอง driver ใน WebStorm ใช้กับ editor เท่านั้น ไม่เกี่ยวกัน ติดตั้ง mysql2 ในโปรเจกต์ ใช้ terminal ใน IDE จะอยู่ในโฟลเดอร์โปรเจกต์อยู่แล้ว

project root

```
$ npm install mysql2
added 12 packages, and audited 13 packages
# a node_modules folder appears, and package.json lists the dependency
```

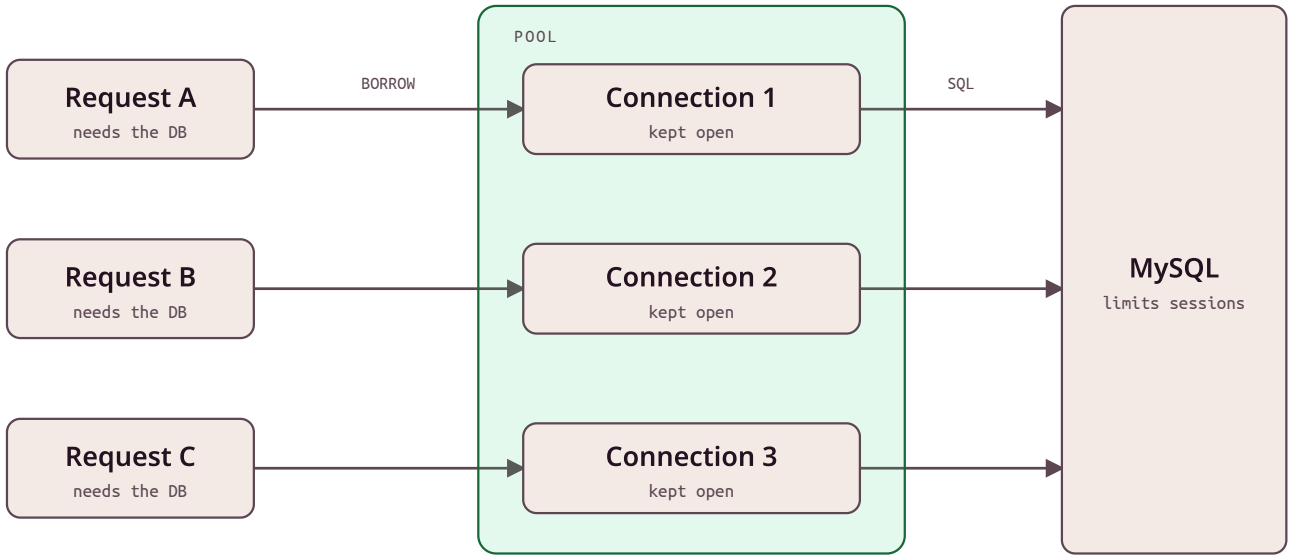
- ใช้ driver ที่ตรงกับ database และเวอร์ชัน ถ้าผิดรุ่นอาจเชื่อมต่อไม่ได้ เราใช้ mysql2 กับ MySQL ส่วน PostgreSQL และ MongoDB มี driver ของตัวเอง
- driver เป็น **async** และ non-blocking จึงเข้ากับ **event loop**

Connection

สร้าง connection ใหม่ทุก **request** ถ้า **client** เป็นร้อย database อาจรับไม่ไหว

Pool

สร้างไว้เช่น 8 connection แล้วให้ request ผลัดกันใช้ ใช้เสร็จไม่ปิด ถ้า 8 ไม่พอ **pool** สร้างเพิ่มให้ ใช้ pool เมื่อขึ้น production



ทั้งสองได้ object ที่มีฟังก์ชัน **query** เหมือนกัน ใส่ pool ไว้ในไฟล์ของมันเอง ไฟล์ไหนก็ import ไปใช้ได้

db/pool.js

```
import * as mysql from 'mysql2/promise';
export const pool = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: 'your-password',
  database: 'sampledb',
});
export default pool;
```

ไฟล์ทดสอบสั้น ๆ เพื่อดูว่า query คืนอะไรก่อนต่อกับส่วนอื่น

tests/test-db.js

```
import pool from '../db/pool.js';
export async function findAll() {
  const subjects = await pool.query('SELECT * FROM subjects');
  return subjects;
}
const [subjects, tableSchema] = await findAll();
await pool.end(); // a test script must close the pool, or it never exits
console.log(subjects);
console.log(tableSchema);
```

SELECT | INSERT | UPDATE | DELETE

```
await pool.query('SELECT * FROM subjects')
```

Index 0

rows

array ของ object หนึ่งต่อหนึ่งแถว

ได้ค่า

```
[ [ { id: 1, subject_code: 'INT 100',
```

ใช้ index 0 ผลที่เจอแค่หนึ่งก็ยังเป็น array ที่มีหนึ่งรายการ และถ้าไม่เจอจะเป็น array ว่าง

- SELECT ได้แถวเป็น JavaScript object ให้เลย ไม่ต้องแปลงแถวของตารางเอง

- ส่วนใหญ่ใช้แค่ index 0 และทิ้ง schema: const [rows] = **await** pool.query(...)
- สคริปต์ทดสอบรันค้างเพราะ pool ยังเปิดอยู่ ปิดด้วย pool.end() ในแอปจริงให้เปิดไว้ตลอดอายุของ **server**

Callback: ฟังก์ชันซ้อนฟังก์ชัน จึงยาวกว่า

callback style

```
pool.query(
  'SELECT * FROM students WHERE age > ?', [18],
  (err, results, fields) => {
    if (err) {
      console.error('Query error:', err);
      return;
    }
    console.log('Results:', results);
  }
);
```

Promise + await: สั้นกว่า และเป็นแบบที่เราใช้ ฟังก์ชันที่มี await ต้องเป็น async

async / await

```
async function main() {
  const [rows] = await pool.query('SELECT * FROM students WHERE age > ?', [18]);
  console.log(rows);
}
```

03

Prepared Statements

โปรแกรมไม่ได้เลือกค่า age front end เป็นคนส่งมา ตอนเขียน SQL จึงยังไม่รู้ค่า เครื่องหมาย ? คือที่ว่าง

- SQL ที่มี ? เป็น **placeholder** คือ **prepared statement** เตรียมไว้ก่อน ใส่ค่าทีหลัง
- ค่าส่งเป็น array ใน argument ที่สอง ใส่แทน ? ตามตำแหน่ง ค่าแรกเข้า ? ตัวแรก และต่อไปเรื่อย ๆ ถ้ามี ? สี่ตัวต้องมีสี่ค่า

- ส่งตัวแปรได้ เช่น id ที่มาจาก request

ค่าที่ส่ง `18` `0 OR 1=1`

ต่อเข้าไปในข้อความ

```
'SELECT * FROM students WHERE age > '
```

database ได้รับ:

```
SELECT * FROM students WHERE age > 18
```

ใช้ได้ ครั้งนี้

Prepared statement

```
pool.query('SELECT * FROM students WHE
```

คำสั่งกับค่าถูกส่งแยกกัน:

```
? = "18"
```

ผลเหมือนกันและปลอดภัย

ทำไมสำคัญ

สไลด์ระบุเป้าหมายว่า "ใช้ prepared statement เพื่อป้องกัน **SQL injection**" การต่อข้อมูลจากผู้ใช้เข้าไปในข้อความ SQL ทำให้ข้อมูลนั้นเปลี่ยนคำสั่งได้ ส่วน placeholder ส่งค่าแยก ค่าจึงยังเป็นแค่ค่า ห้ามสร้าง SQL ด้วยการต่อ string กับข้อมูลจาก request

04

The Repository

ตรงนี้ layer เห็นผล เปลี่ยนแค่ data access layer จาก array เป็น database ส่วน **service** และ **router** ด้านบนไม่ต้องแก้

repositories/subject-repository.js

```
import pool from '../db/pool.js';
export async function findAll() {
  const [subjects] = await pool.query('SELECT * FROM subjects');
  return subjects;
}
export async function findById(id) {
  const [subject] = await pool.query('SELECT * FROM subjects WHERE id = ?', [id]);
  if (subject.length == 0) return null;
  return subject;
}
export async function create(subject) {
  const [result] = await pool.query(
    'INSERT INTO subjects (subject_code, subject_title, credit) VALUES (?, ?, ?)',
    [subject.code, subject.title, subject.credit]);
  if (result.affectedRows === 0) return false;
  return findById(result.insertId);
}
export async function update(subject) {
  const [result] = await pool.query(
    'UPDATE subjects SET subject_code = ?, subject_title = ?, credit = ? WHERE id = ?',
    [subject.code, subject.title, subject.credit, subject.id]);
  if (result.affectedRows === 0) return false;
  return findById(subject.id);
}
export async function remove(id) {
  const [result] = await pool.query('DELETE FROM subjects WHERE id = ?', [id]);
  return result.affectedRows > 0;
}
```

- findById ยังได้ array แม้แถวเดียว code นี้ return array นั้น ถ้าไม่มีแถวคืนค่า null ถ้าอยากได้ object เดียวให้ใช้ [0]
- create ไม่ส่ง id หลัง insert จะอ่านแถวใหม่กลับมาด้วย **insertId**
- update เช็ค **affectedRows** ถ้าเป็น 0 แปลว่าไม่มีแถวที่ id นั้น
- อาจารย์ชอบให้ update ค้นแถวก่อน ถ้าไม่มี return false และอ่านแถวอีกครั้งเพื่อยืนยันว่าเปลี่ยนจริง

ผลกระทบของ await

เมื่อฟังก์ชันใน **repository** เป็น async ทุกที่ที่เรียกต้อง await ถ้าลืมจะได้ **Promise** ที่ค้างอยู่ หรือแถวใหม่ไม่โผล่ ให้แก้ทีละ layer: พิมพ์ผลที่ repository แล้ว service แล้ว router เพื่อหาว่าพังที่ไหน

05

Reading the Body and Writing the Response

POST หรือ PUT พก JSON **body** มา ถ้ามีแค่ http **module** ต้องเก็บ body เป็นชิ้น ๆ แล้ว parse เอง จากนั้นเขียนคำตอบ JSON ด้วย helper ตัวเดียว

router.js: getBody and writeResponse

```
function getBody(request) {
  return new Promise((resolve, reject) => {
    let body = '';
    // collect chunks as they arrive, stream by stream
    request.on('data', chunk => { body += chunk.toString(); });
    // the full payload has arrived: parse the text into an object
    request.on('end', () => resolve(body ? JSON.parse(body) : null));
    request.on('error', err => reject(err));
  });
}
async function writeResponse(response, status, message) {
  response.writeHead(status, { 'Content-Type': 'application/json' });
  response.end(JSON.stringify(message));
}
```

router.js: createNewSubject

```
async function createNewSubject(request, response) {
  const subject = await getBody(request, response);
  let message = null;
  let status = 201;
  if (subject) {
    const newSubject = await service.createNewSubject(subject);
    message = newSubject;
    if (!newSubject) {
      status = 409;
      message = { error: `Duplicate subject for id ${subject.id}` };
    }
  } else {
    message = { error: 'Bad request, body is empty' };
    status = 400;
  }
  writeResponse(response, status, message);
}
```

- body ว่าง: 400 Bad Request ซ้ำ: 409 Conflict สร้างสำเร็จ: 201
- ไม่เจอตอน update: 404
- subject_code ซ้ำจะถูก database ปฏิเสธเองด้วย duplicate entry error เรายังไม่ได้ดักข้อผิดพลาดนี้ โปรแกรมจึงตาย เรื่อง **error handling** อยู่คลาสถัดไป
- ใส่ **header** Content-Type และพิมพ์ข้อความช่วย **debug** จะได้รู้ว่าเกิดอะไรขึ้นแม้ตอบกลับมาว่าง

06

CRUD over HTTP

ทุกอย่างเชื่อมกัน **endpoint** กำหนด SQL, repository รันมัน และ **status code** บอกผล

GET /subjects

`SELECT * FROM subjects`

`findAll()` -> 200

GET /subjects/:id

`SELECT ... WHERE id = ?`

`findById(id)` -> 200 or 404

POST /subjects

`INSERT INTO subjects ...`

`create(subject)` -> 201 or 409

PUT /subjects/:id

`UPDATE subjects SET ... WHERE id = ?`

`update(subject)` -> 200 or 404

DELETE /subjects/:id

`DELETE FROM subjects WHERE id = ?`

`remove(id)` -> 204 or 404

- การบ้านจากอาจารย์: เขียน PUT เอง ถ้า create ใช้ได้แล้ว update ต่างแค่ SQL กับ id
- เก็บโปรเจกต์ไว้ คลาสถัดไปต่อยอดจากอันเดิม

07

อภิธานศัพท์

affectedRows
: จำนวนแถวที่ INSERT, UPDATE หรือ DELETE เปลี่ยน ใช้เช็คว่าคำสั่งสำเร็จไหม

async
: เริ่มงานที่ช้า แล้วทำงานอื่นต่อ และถูกเรียกกลับเมื่อเสร็จ แทนที่จะรอ

AUTO_INCREMENT
: database นับ id เพิ่มให้เอง ไม่ต้องส่ง id ตอน INSERT และไม่ซ้ำ

await
: หยุด async function นี้จนกว่า Promise จะเสร็จ ใช้ได้เฉพาะใน async function

back end

ส่วนบน server จัดการข้อมูล กฎ และ database แล้วตอบ front end ด้วย JSON

body

ข้อมูลที่ request พกมาหลัง header สำหรับ API มักเป็น JSON

client

ฝั่งที่ร้องขอ มักเป็น browser แต่เป็นโปรแกรมหรืออุปกรณ์อะไรก็ได้

connection

ช่องทางที่เปิดอยู่ระหว่างโปรแกรมกับ database ต้องมีก่อนส่ง SQL ได้

CRUD

Create, Read, Update, Delete: สี่อย่างพื้นฐานที่ทำกับข้อมูล

DBMS

ระบบจัดการฐานข้อมูล เช่น MySQL back end ส่งคำสั่ง SQL ไปให้

debug

หาและแก้บั๊ก มักด้วยการไล่ code ทีละบรรทัดและดูค่า

driver

library ที่รู้วิธีคุยกับ database ชนิดหนึ่ง สำหรับ MySQL ใน Node คือ mysql2

endpoint

method กับ path หนึ่งคู่ที่ API ตอบ เช่น GET /api/subjects

error handling

ดัก error เพื่อให้โปรแกรมไม่ตายและ client ได้คำตอบที่มีประโยชน์

event loop

ส่วนของ Node ที่หยิบ request จาก event queue ทีละอัน และส่งคำตอบที่เสร็จแล้วกลับไป

front end

ส่วนที่ผู้ใช้เห็นและใช้ รันบน browser หรือโทรศัพท์

header

ข้อมูลเสริมของข้อความ ส่งเป็นบรรทัด name: value เช่น Content-Type

insertId

id ที่ database ให้แถวที่เพิ่ง insert ด้วย AUTO_INCREMENT

JSON

JavaScript Object Notation: รูปแบบข้อมูลที่เป็นข้อความ คนและโปรแกรมอ่านได้

localhost

ชื่อของเครื่องตัวเอง localhost:3000 คือ port 3000 บนเครื่องนี้

module

ไฟล์ code ที่โหลดเข้าไฟล์อื่นด้วย import

Node

runtime ที่รัน JavaScript นอก browser ทำให้ใช้ภาษาเดียวกันเขียน back end ได้

placeholder

เครื่องหมาย ? ใน prepared statement ค่าจะถูกใส่แทน ? ตามลำดับ

pool

การเชื่อมต่อที่เปิดค้างไว้ให้ใช้ร่วมกัน request ยืมไปใช้แล้วคืน database จึงไม่ถูกท่วม

port

เลขที่บอกว่าโปรแกรมไหนบนเครื่องควรได้รับข้อความ web server ใช้ 80 เป็นค่าเริ่มต้น

prepared statement

SQL ที่เขียน ? ไว้ตรงที่จะใส่ค่า แล้วส่งค่าแยกต่างหาก database จะไม่เข้าใจผิดว่าค่าเป็นคำสั่ง

primary key

คอลัมน์ที่ระบุแต่ละแถว มักเป็น id การ update และ delete ต้องใช้

Promise

object แทนผลลัพธ์ที่ยังไม่พร้อม await รอมัน

query

ส่วนของ URL หลัง ? เช่น page=2&size=10 ใช้กรองหรือแบ่งหน้ารายการ

repository

layer ที่อ่านเขียน database ไม่มี business rule และไม่มี HTTP

request

สิ่งที่ client ส่งมา: method, URL, header และบางครั้งมี body

router

การจับคู่ HTTP method และ URL path กับฟังก์ชันที่จัดการ

server

โปรแกรม (หรือเครื่องที่รันมัน) ที่รอรับ request แล้วตอบกลับ

service

layer ที่เก็บ business rule เรียก repository และไม่รู้เรื่อง HTTP

SQL injection

การโจมตีที่ข้อมูลจากผู้ใช้ถูกต่อเข้า SQL แล้วเปลี่ยนความหมายของคำสั่ง placeholder ป้องกันได้

status code

ตัวเลขที่บอก client ว่าผลเป็นอย่างไร เช่น 200 OK, 201 Created, 404 Not Found, 500 Server Error

surrogate key

key ที่สร้างขึ้นไม่มีความหมาย เช่น เลขรัน ง่ายกว่าใช้รหัสจริง